Quicksort

Algorithms § 2.3

Agenda

- **From 30,000 feet:** pick a pivot, partition around it, repeat
- **Partitioning in place:** the i/j scan (the part to really get)
- **A full trace:** the rest of the sort, call by call
- **Why we shuffle first:** what goes wrong on already-sorted input
- **Stability:** what quicksort gives up

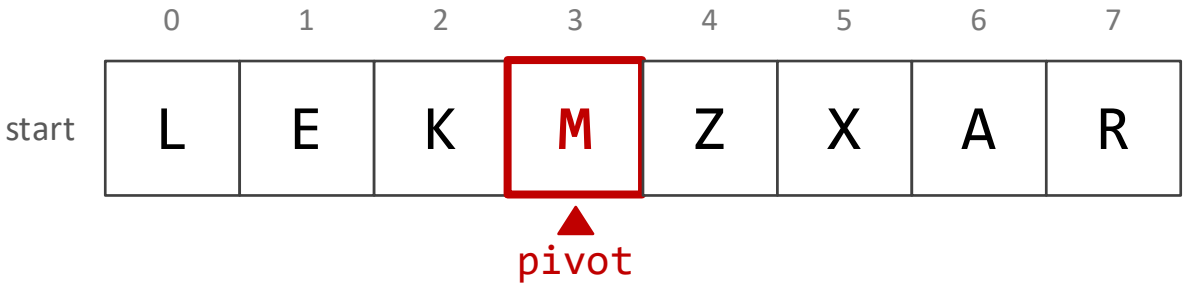
Why another n log n sort?

Mergesort already guarantees n log n. Quicksort's advantages are elsewhere:

	mergesort	**quicksort**
extra memory	aux: n items	**none: sorts in place**
speed in practice	fast	**usually faster (next time: why)**
worst case	≤ n lg n compares	**~$n^2/2$ compares**
stable?	yes (ties go left)	**no**

That worst case is real, but it's astronomically unlikely, because quicksort uses **randomness**. So we describe quicksort by its **average-case** (expected) running time: the average over its own random choices, for any input you give it.

Quicksort from 30,000 feet



quicksort, roughly

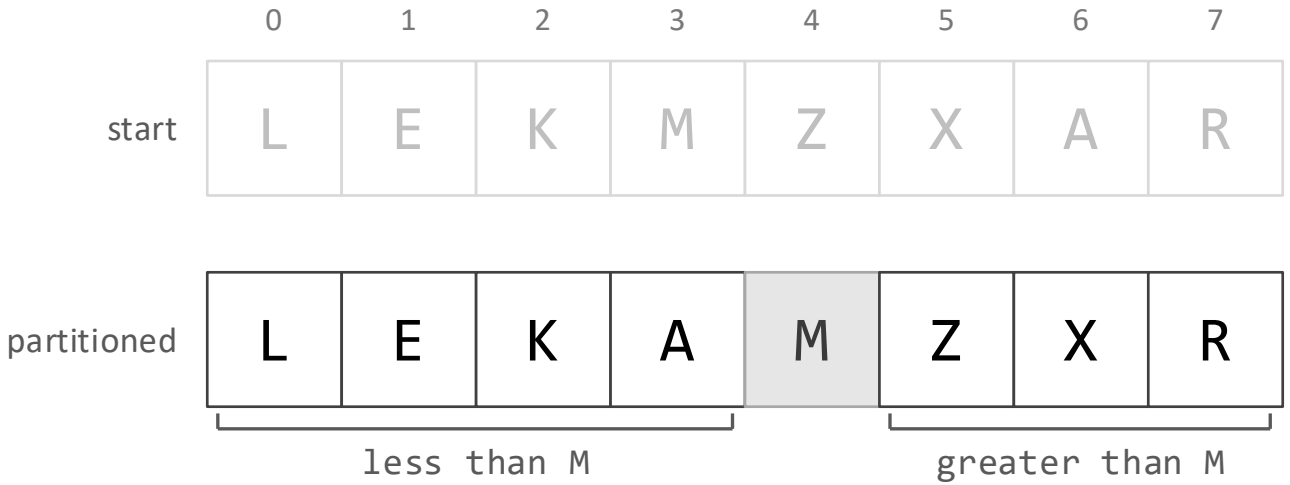
1. Pick a random item: the **pivot**

2. **Partition:** smaller items to its left, bigger to its right

3. **Repeat** on each side, until every piece has size 0 or 1

Step 1: pick an item at random and call it the **pivot**. Here the random pick was index 3, so the pivot is **M**.

Quicksort from 30,000 feet



quicksort, roughly

1. Pick a random item: the **pivot**

2. Partition: smaller items to its left, bigger to its right

3. **Repeat** on each side, until every piece has size 0 or 1

Step 2: partition. Everything smaller than M goes to its left, everything bigger to its right. **M is now in its final position**, and it never moves again.

The two sides are **not** sorted. All we know is which side of M each item is on.

Quicksort from 30,000 feet



quicksort, roughly

1. Pick a random item: the **pivot**

2. **Partition:** smaller items to its left, bigger to its right

3. Repeat on each side, until every piece has size 0 or 1

Step 3: repeat on each side. Random picks: **E** on the left, **R** on the right. Partition each piece the same way. R happens to be the smallest on its side, so its left piece is empty. A is a piece of size 1: already sorted.

Quicksort from 30,000 feet



quicksort, roughly

1. Pick a random item: the **pivot**

2. **Partition:** smaller items to its left, bigger to its right

3. Repeat on each side, until every piece has size 0 or 1

Pivots **K** and **X**, and every piece is down to size 0 or 1. **A E K L M R X Z: sorted.**

Quicksort has no combine step

Mergesort

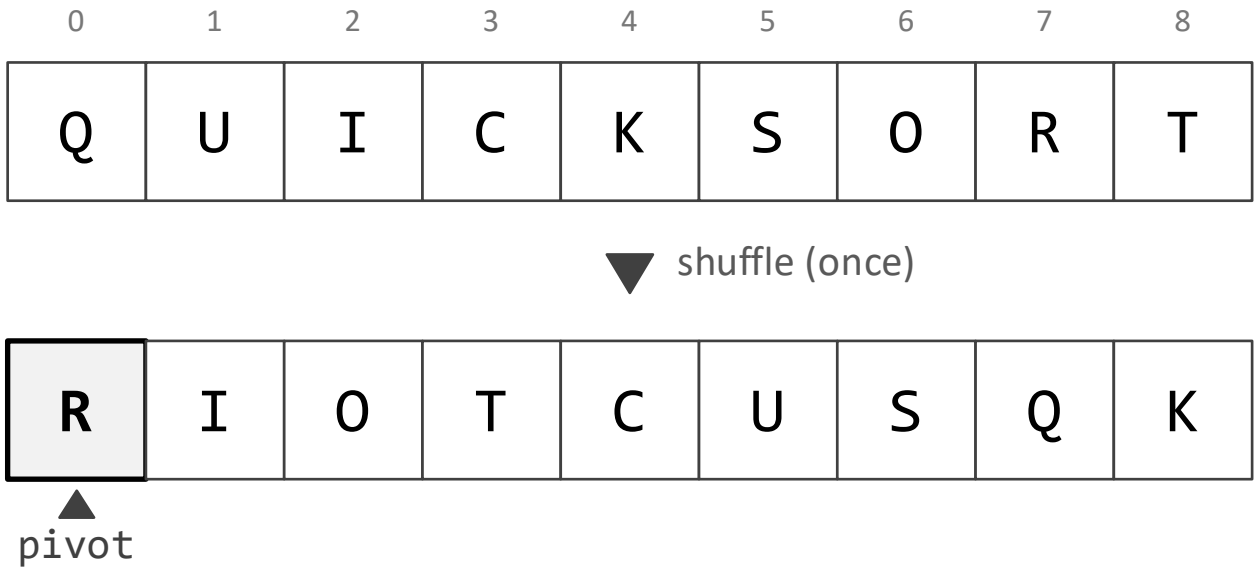
1. **Split:** find the middle (no real work)
2. **Sort each half** (recursive calls)
3. **Merge** the sorted halves (**all the real work**)

Quicksort

1. **Partition** around the pivot (**all the real work**)
2. **Sort each side** (recursive calls)
3. **Combine:** nothing to do

In mergesort the work happens **after** the recursive calls. In quicksort it happens **before** them. Once every piece is size 0 or 1, the array is already sorted, right where it is.

Shuffle once, then take the leftmost

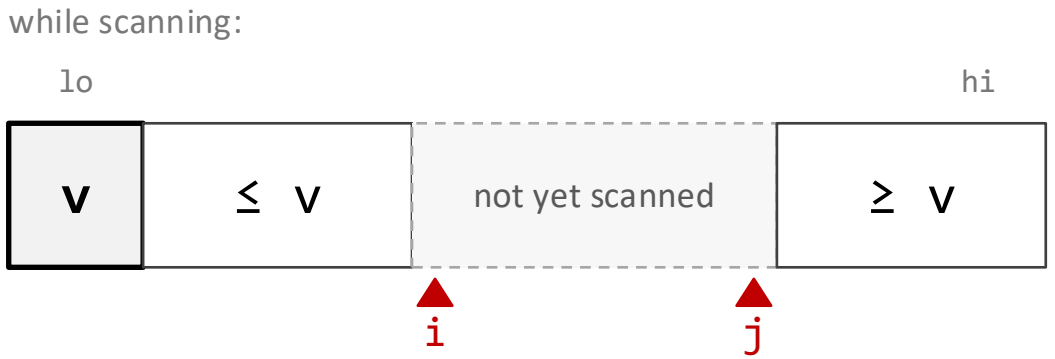


- Mergesort needed a second array, **aux**. Quicksort doesn't: it partitions by **swapping items inside a**.
- After a random shuffle, every item is equally likely to be in **a[0]**. So taking the leftmost item **is** picking at random.
- Partitioning leaves each piece in random order too, so this stays true in every recursive call.

Why bother shuffling at all? We'll see what goes wrong without it after the full trace.

Partitioning in place: the rules

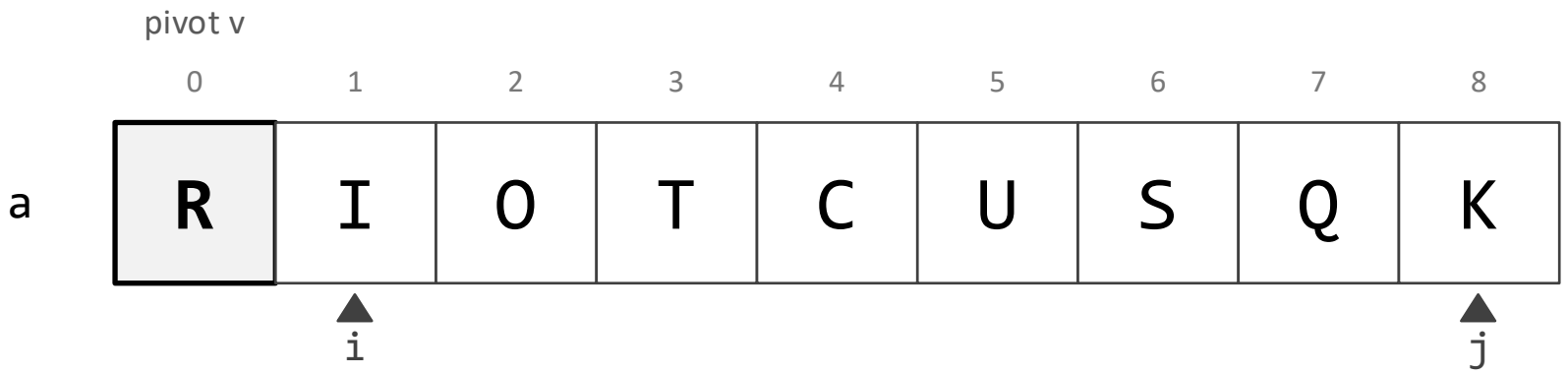
1. The pivot is **v = a[lo]**. **i** starts just right of it; **j** starts at the last item, **a[hi]**.
2. **Scan with i:** move right until a[i] is **not less** than v.
3. **Scan with j:** move left until a[j] is **not greater** than v.
4. If i and j have **crossed** (i ≥ j), go to step 5. Otherwise **swap a[i] and a[j]**, step each pointer once more, and go back to step 2.
5. **Swap the pivot a[lo] with a[j].** Now v is in its final position.



Everything between the pivot and i is ≤ v. Everything right of j is ≥ v. Each swap grows both sides, until there's nothing left in the middle.

Both pointers **stop on items equal to v**. That looks like wasted work. Next time we'll see why it isn't.

Partition: trace



cost so far

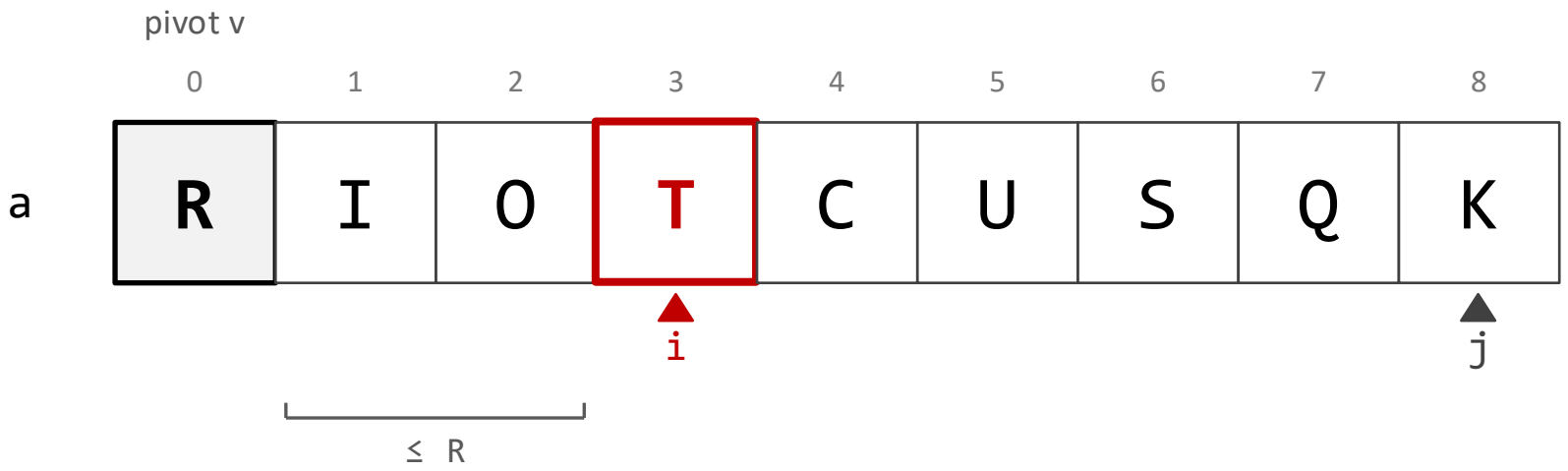
compares	0
exchanges	0

nothing compared yet

`sort(a, 0, 8)`. The pivot is **R**, the leftmost item. **i** starts at a[1], just right of the pivot. **j** starts at a[8], the last item.

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Partition: trace



cost so far

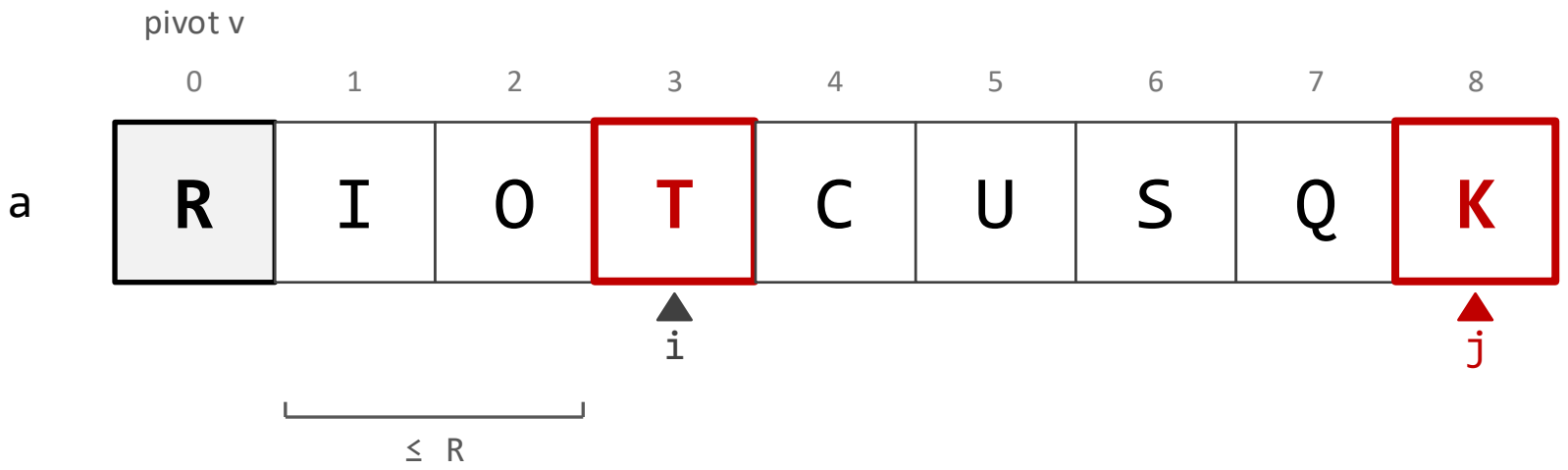
compares	3
exchanges	0

i: I, O, T

i moves right past **I** and **O** (both less than R), and stops at **T**: T is **not less** than R.

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Partition: trace



cost so far

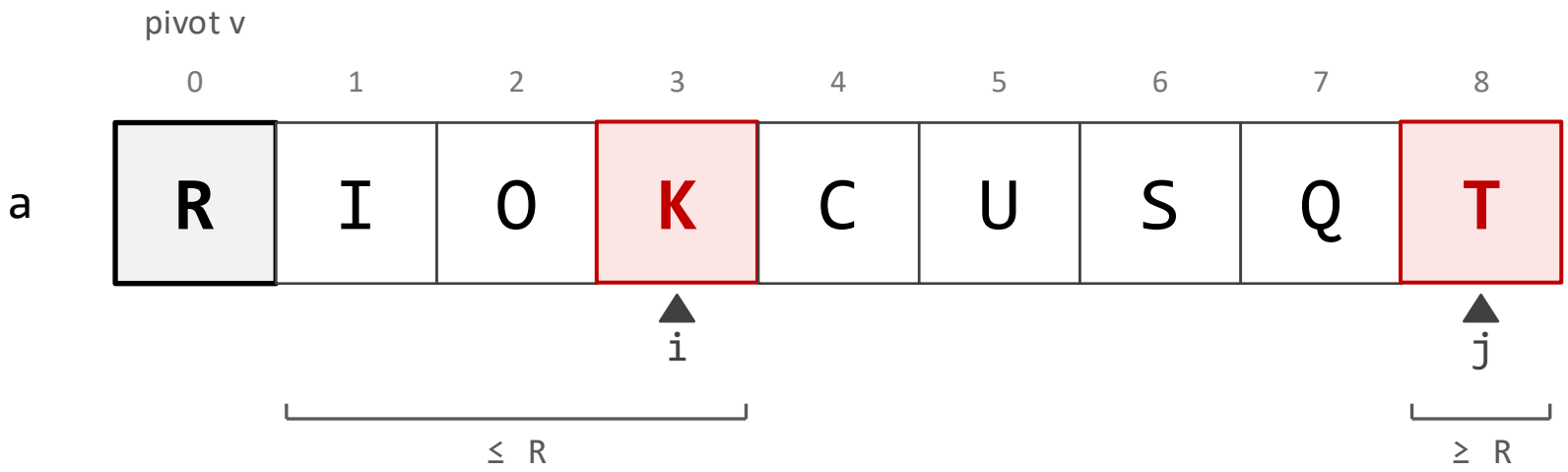
compares	4
exchanges	0

j: K

j checks **K**. K is **not greater** than R, so j stops right where it started.

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Partition: trace



cost so far

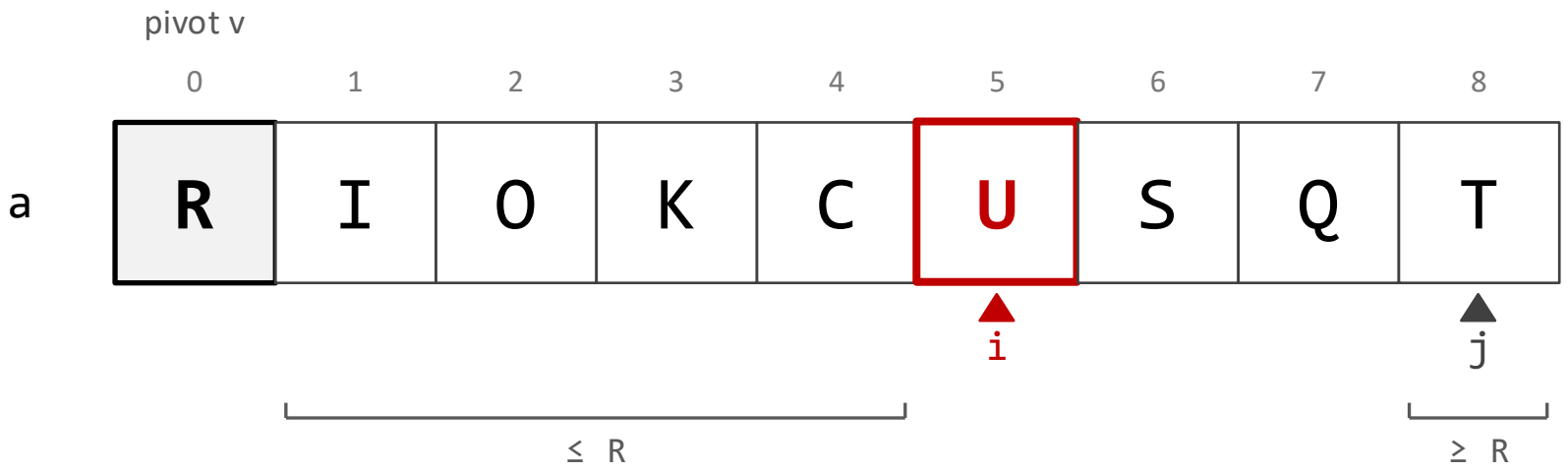
compares	4
exchanges	1

swap T and K

i and j haven't crossed, so **swap a[i] and a[j]**. Now K is on the small side and T is on the big side.

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Partition: trace



cost so far

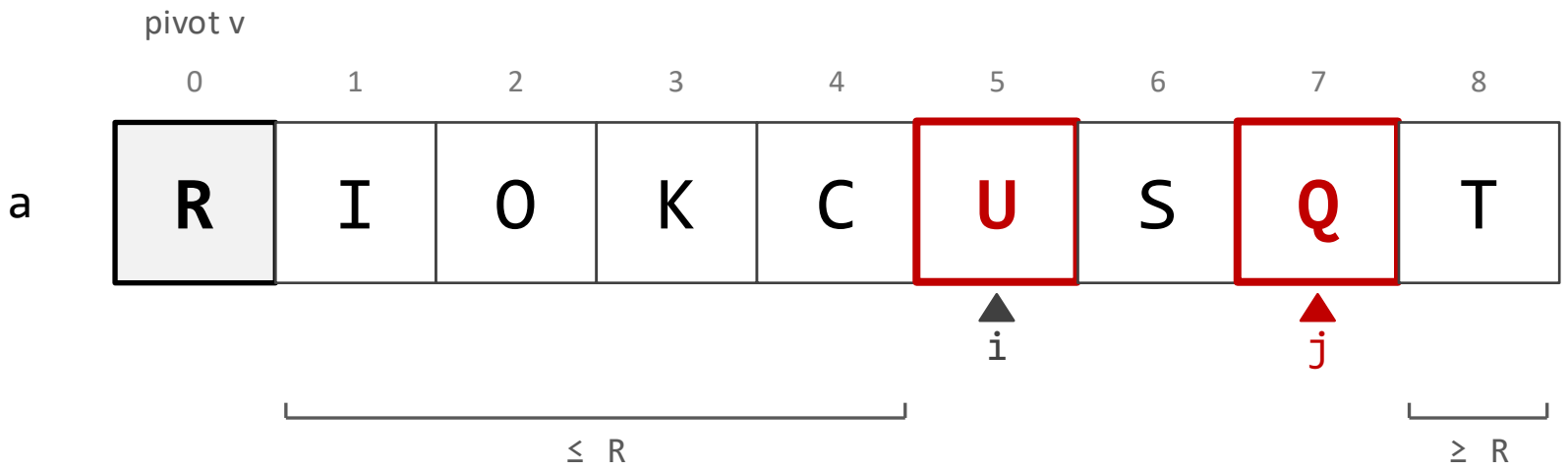
compares	**6**
exchanges	1

i: C, U

Keep scanning. **i** steps past **C** (less than R) and stops at **U**.

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Partition: trace



cost so far

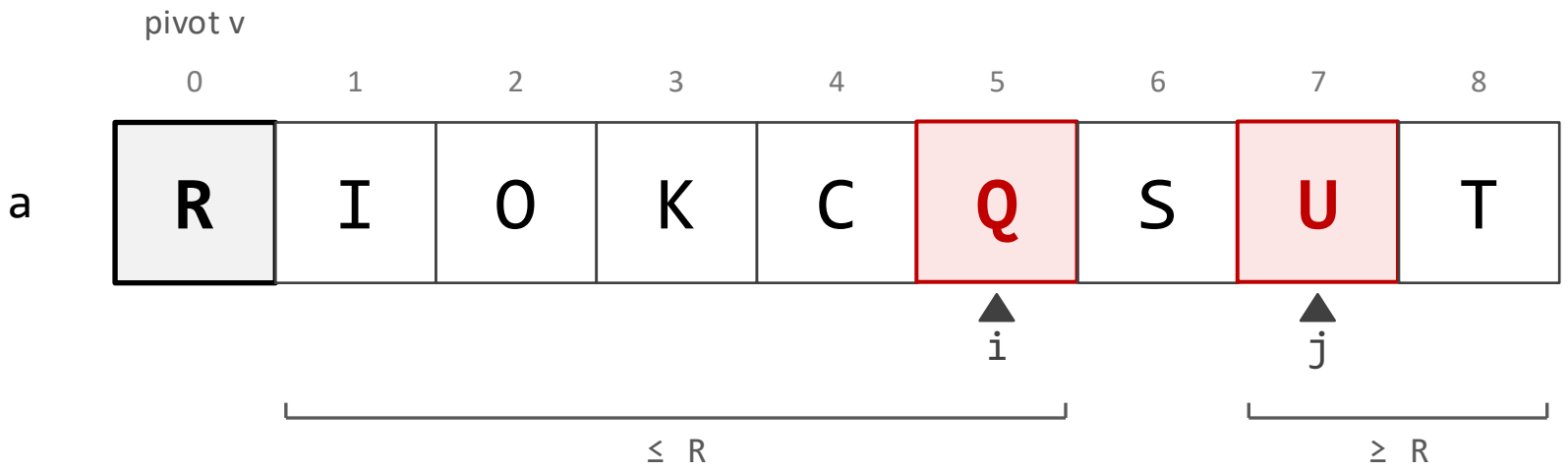
compares	7
exchanges	1

j: Q

j steps left to **Q** and stops: Q is not greater than R.

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Partition: trace



cost so far

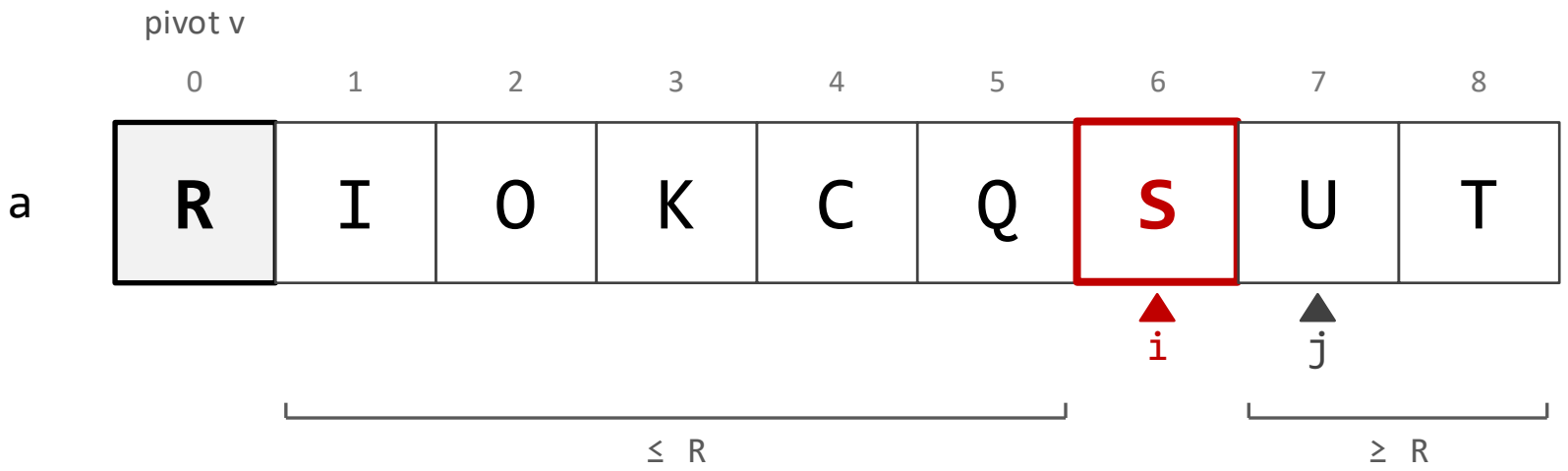
compares	7
exchanges	2

swap U and Q

Not crossed yet, so **swap**: U and Q trade places.

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Partition: trace



cost so far

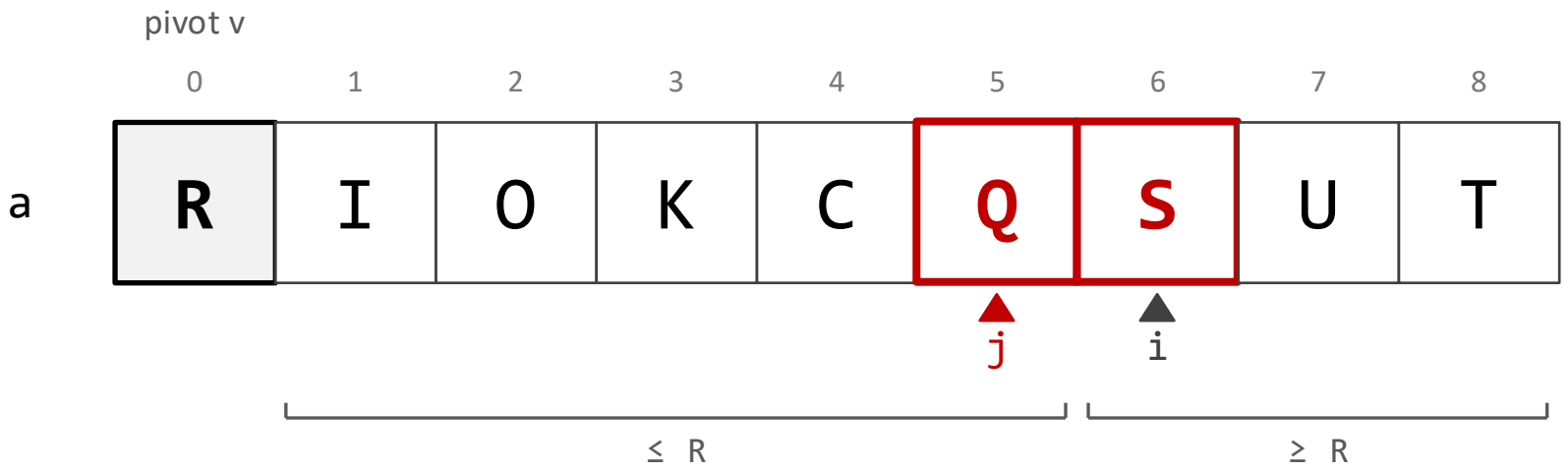
compares	8
exchanges	2

i: S

i steps to **S** and stops right away: S is not less than R.

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Partition: trace



cost so far

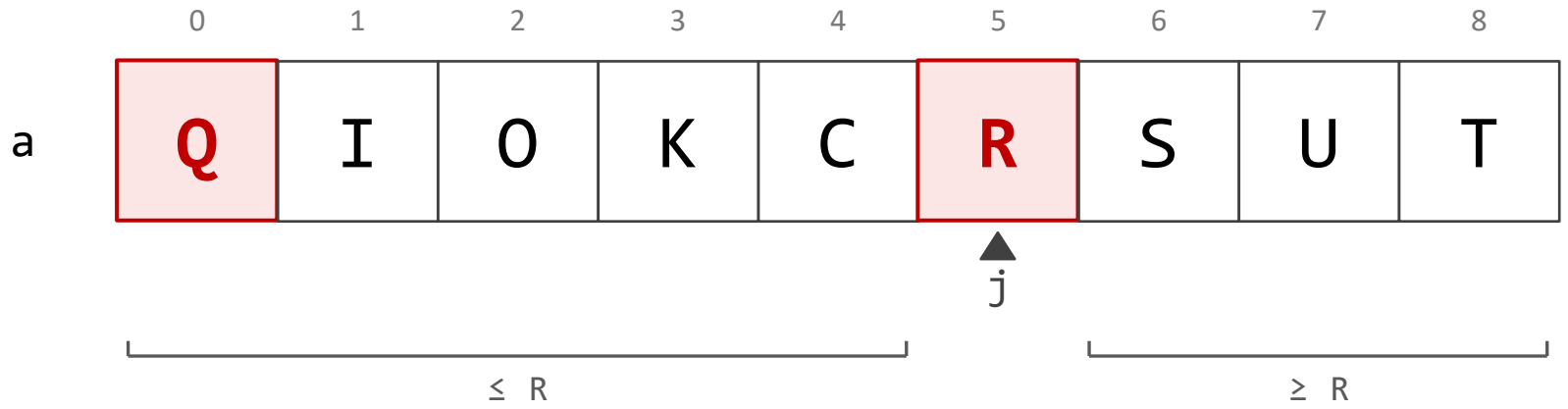
compares	**10**
exchanges	2

j: S, Q

j steps past **S** (greater than R) and stops at **Q**. Now j is **left of** i: the pointers have **crossed**, so scanning is over.

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Partition: trace



cost so far

compares	10
exchanges	3

swap pivot into a[j]

Last step: **swap the pivot with a[j]**. **R is in its final position**: everything to its left is ≤ R, everything to its right is ≥ R. 10 compares, 3 exchanges.

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Partition: pseudocode

```
1  partition(a, lo, hi):
2      v = a[lo]                                   (the pivot)
3      i = lo,  j = hi + 1
4      repeat:
5          do i = i + 1 while a[i] < v and i < hi
6          do j = j - 1 while a[j] > v and j > lo
7          if i >= j: break                        (crossed: stop)
8          swap a[i] and a[j]
9      swap a[lo] and a[j]                         (pivot into place)
10     return j                                    (where the pivot ended up)
```

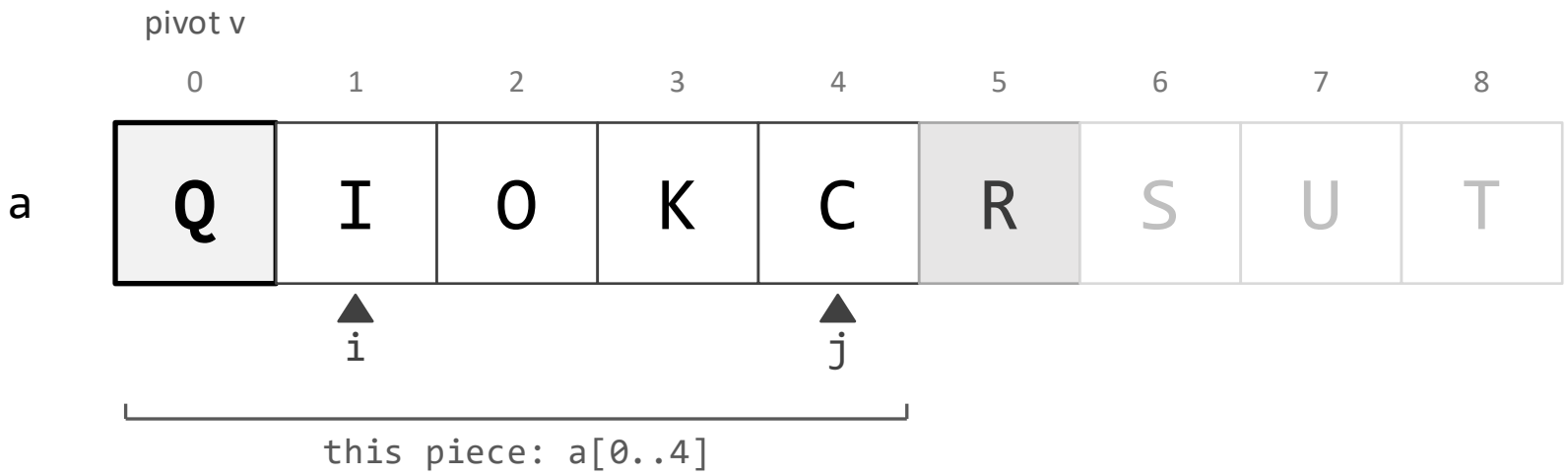
- **Lines 5–6 are the scans.** Each pointer steps once, then keeps going while its item is on the correct side. Both stop on an item **equal** to v.
- **i needs the `i < hi` check**, or it could run off the end of the piece. (You'll see it matter in the next trace.)
- **j's `j > lo` check is never actually needed:** j stops at a[lo] at the latest, because a[lo] is v, and v is not greater than v. But keep it in your code; the book does.

Quicksort: pseudocode

```
1  quicksort(a):
2      shuffle(a)                                  (once, at the start)
3      sort(a, 0, n - 1)
4
5  sort(a, lo, hi):
6      if hi <= lo: return                         (one item or none)
7      j = partition(a, lo, hi)                    (a[j] is now final)
8      sort(a, lo, j - 1)                          (sort the left piece)
9      sort(a, j + 1, hi)                          (sort the right piece)
```

- Compare with mergesort's `sort`: there, the two recursive calls come first and `merge` comes last. Here, `partition` comes first, and there's **nothing left to do** after the calls.
- Neither recursive call includes **j**: the pivot is already where it belongs.

Partition 2: trace



cost so far

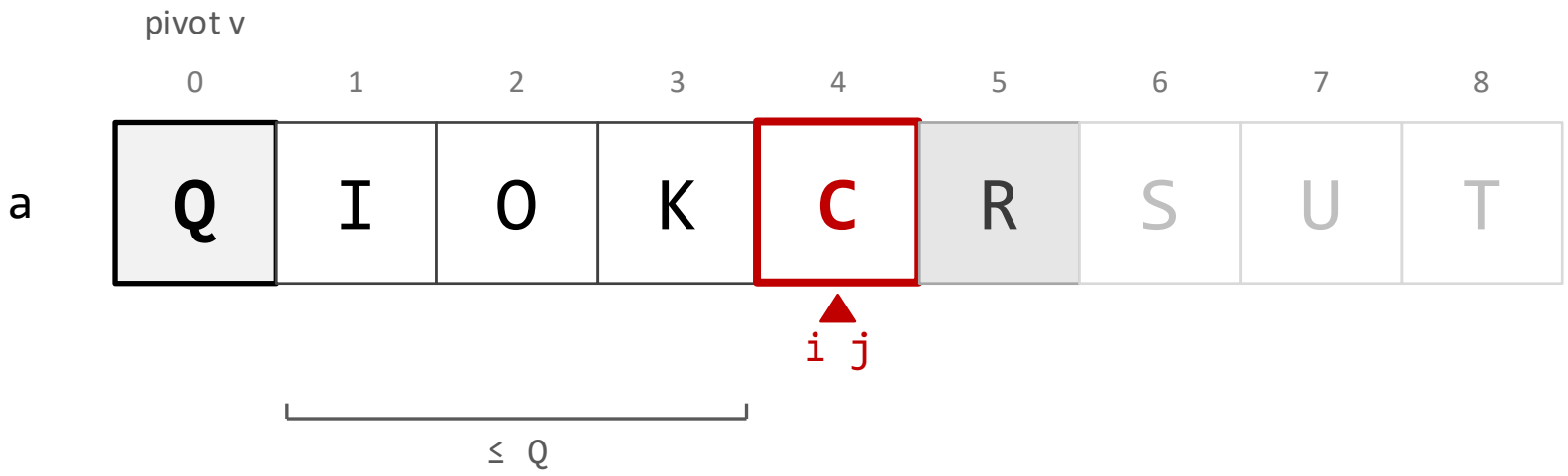
compares	10
exchanges	3

carried over from partition 1

Now the left piece: `sort(a, 0, 4)`. The pivot is **Q**. **i** starts at a[1]; **j** starts at a[4], the **end of this piece** (not the end of the array).

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Partition 2: trace



cost so far

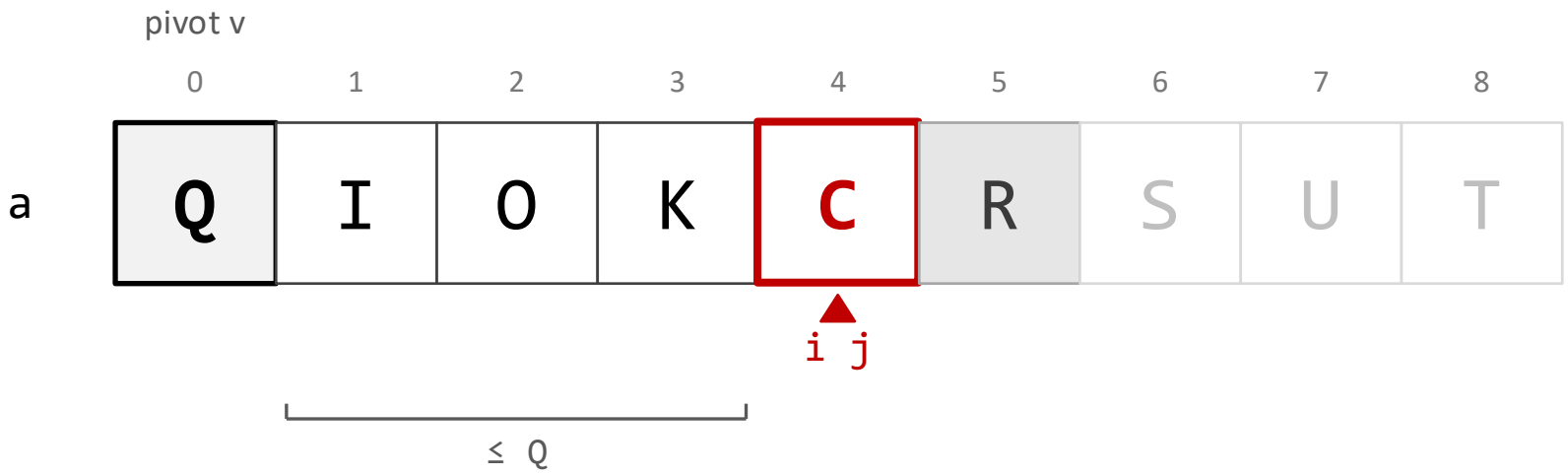
compares	**14**
exchanges	3

i: I, O, K, C

i moves past **I**, **O**, **K**, and reaches **C**. Everything is less than Q, but at a[4] i hits the **end of the piece** (`i == hi`), so it stops there.

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Partition 2: trace



cost so far

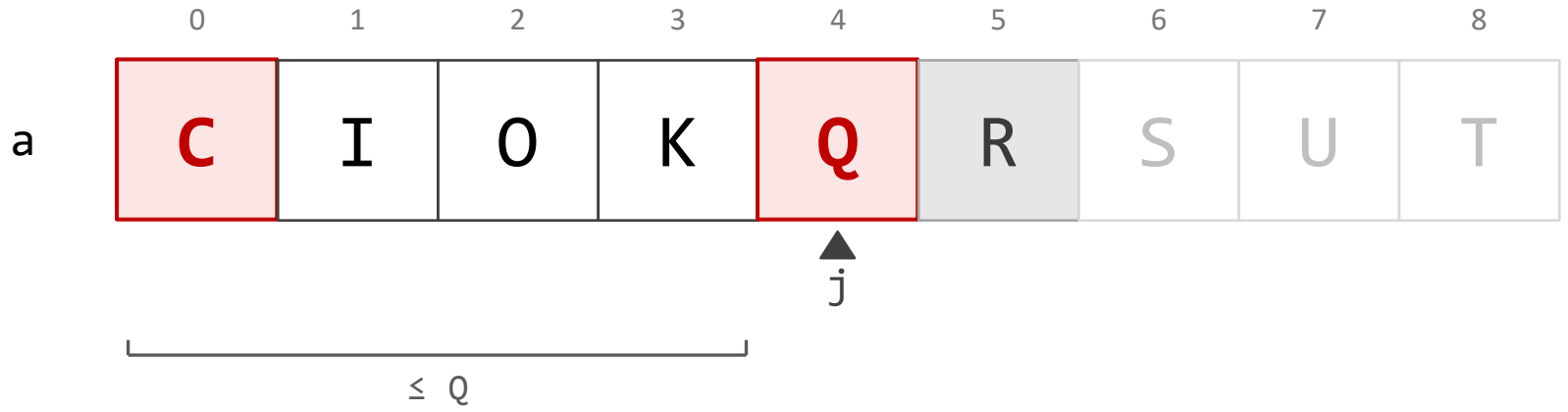
compares	**15**
exchanges	3

j: C

j checks **C**: not greater than Q, so j stops immediately. i and j are on the **same** cell. That counts as crossed (i ≥ j).

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Partition 2: trace



cost so far

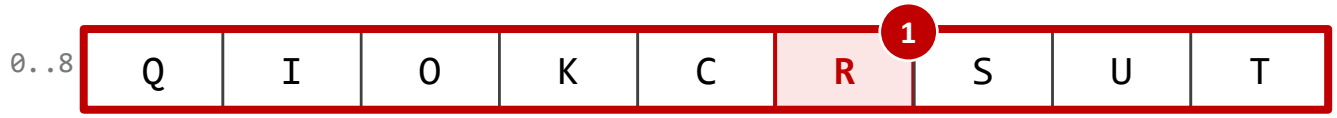
compares	15
exchanges	4

swap pivot into a[j]

Swap the pivot with a[j]. **Q is in its final position**, at the far end of its piece: it was the **largest** item there. Left piece: a[0..3]. Right piece: **empty**.

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

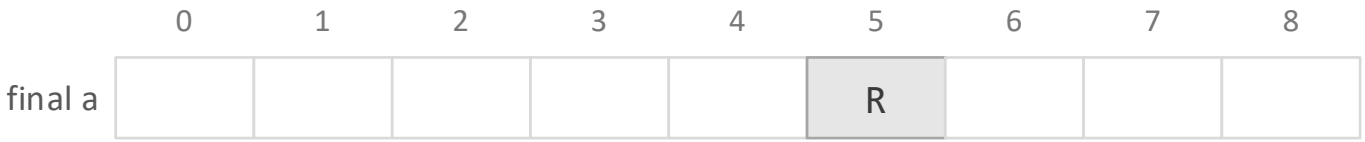
Quicksort: the calls



cost of the whole sort so far

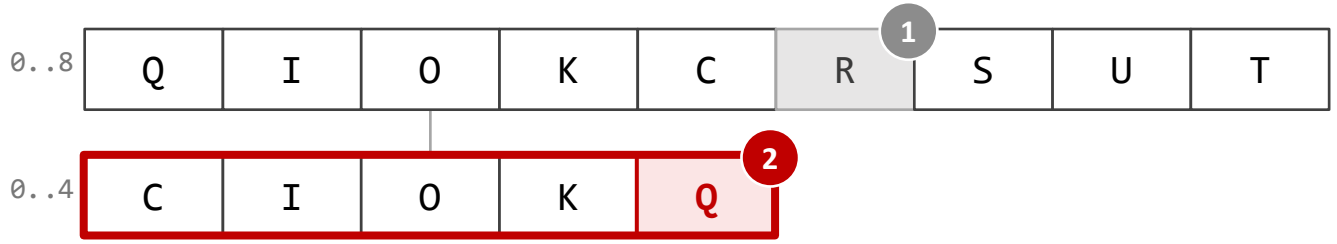
compares	10
exchanges	3

this partition: +10 compares, +3 exch.



Partition 1 (the first trace): `sort(a, 0, 8)`. Pivot **R** lands at a[5]. Each box is one call to `sort`, showing its piece **after** partitioning.

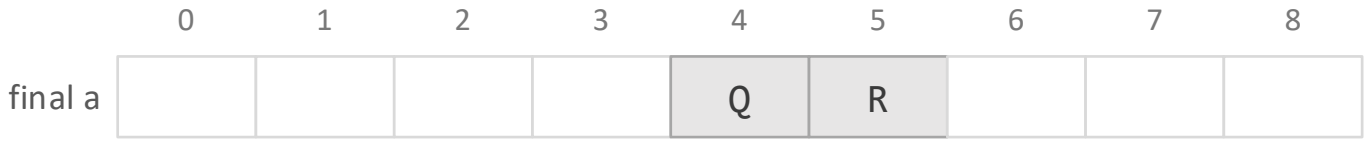
Quicksort: the calls



cost of the whole sort so far

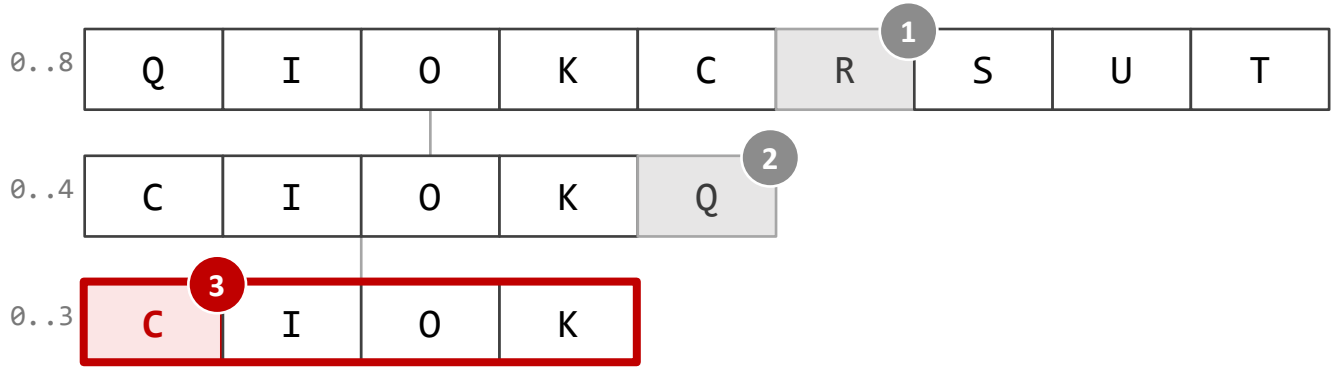
compares	15
exchanges	4

this partition: +5 compares, +1 exch.



Partition 2 (the trace we just did): `sort(a, 0, 4)`. Q was the largest in its piece, so it lands at the end. Its right piece is empty.

Quicksort: the calls



cost of the whole sort so far

compares **20**

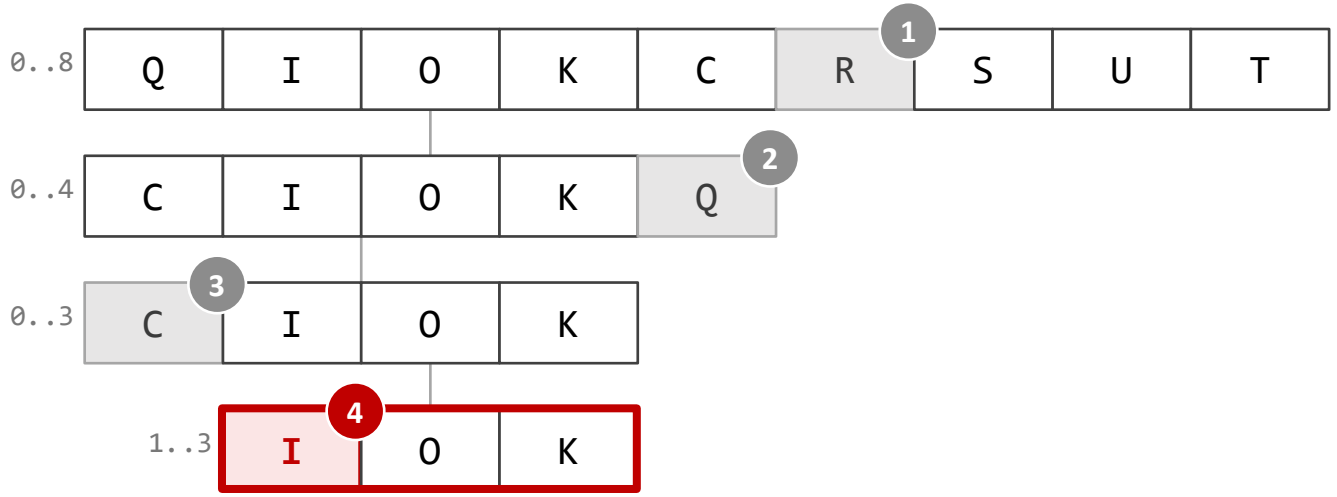
exchanges **5**

this partition: +5 compares, +1 exch.



Partition 3: `sort(a, 0, 3)`. Pivot **C** is the **smallest** in its piece: no left piece, and the right piece is only one item shorter.

Quicksort: the calls



cost of the whole sort so far

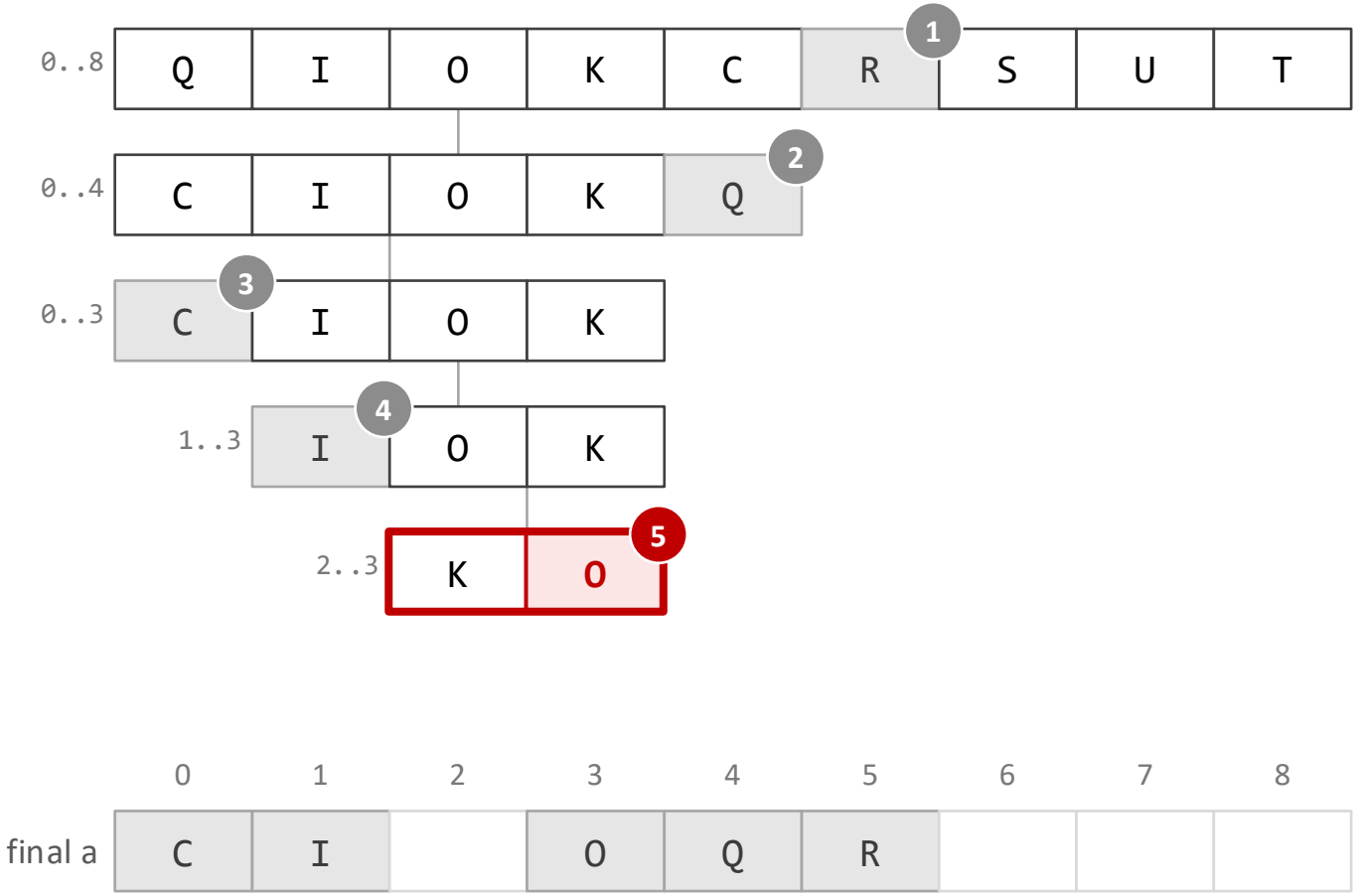
compares	24
exchanges	6

this partition: +4 compares, +1 exch.



Partition 4: `sort(a, 1, 3)`. Pivot **I** is the smallest **again**. Pieces shrinking by just one item is the slow case for quicksort. Remember this shape.

Quicksort: the calls



cost of the whole sort so far

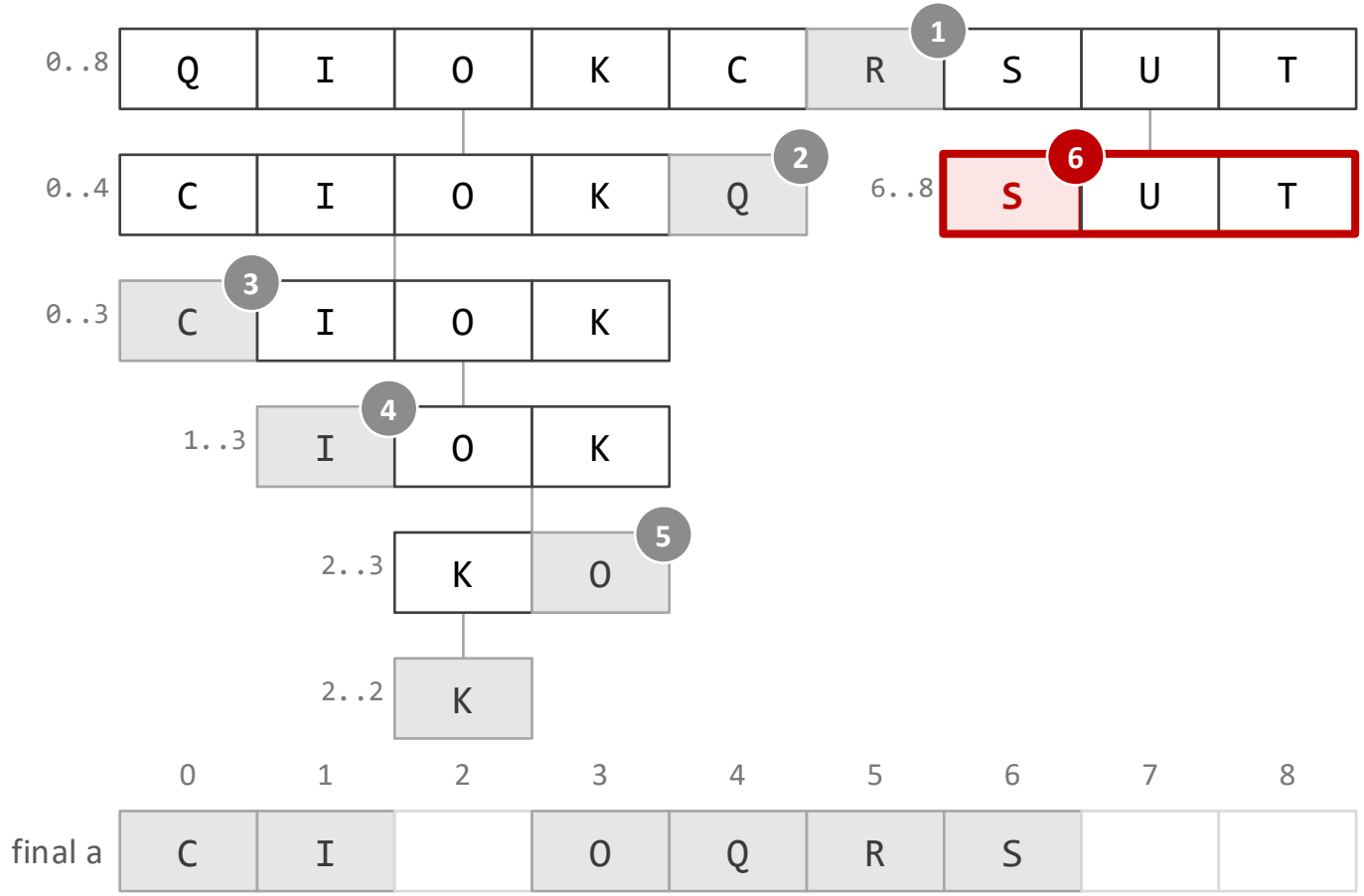
compares **26**

exchanges **7**

this partition: +2 compares, +1 exch.

Partition 5: `sort(a, 2, 3)`. O is bigger than K, so they swap. **K** is left as a piece of size 1: already sorted, so that call returns immediately.

Quicksort: the calls



cost of the whole sort so far

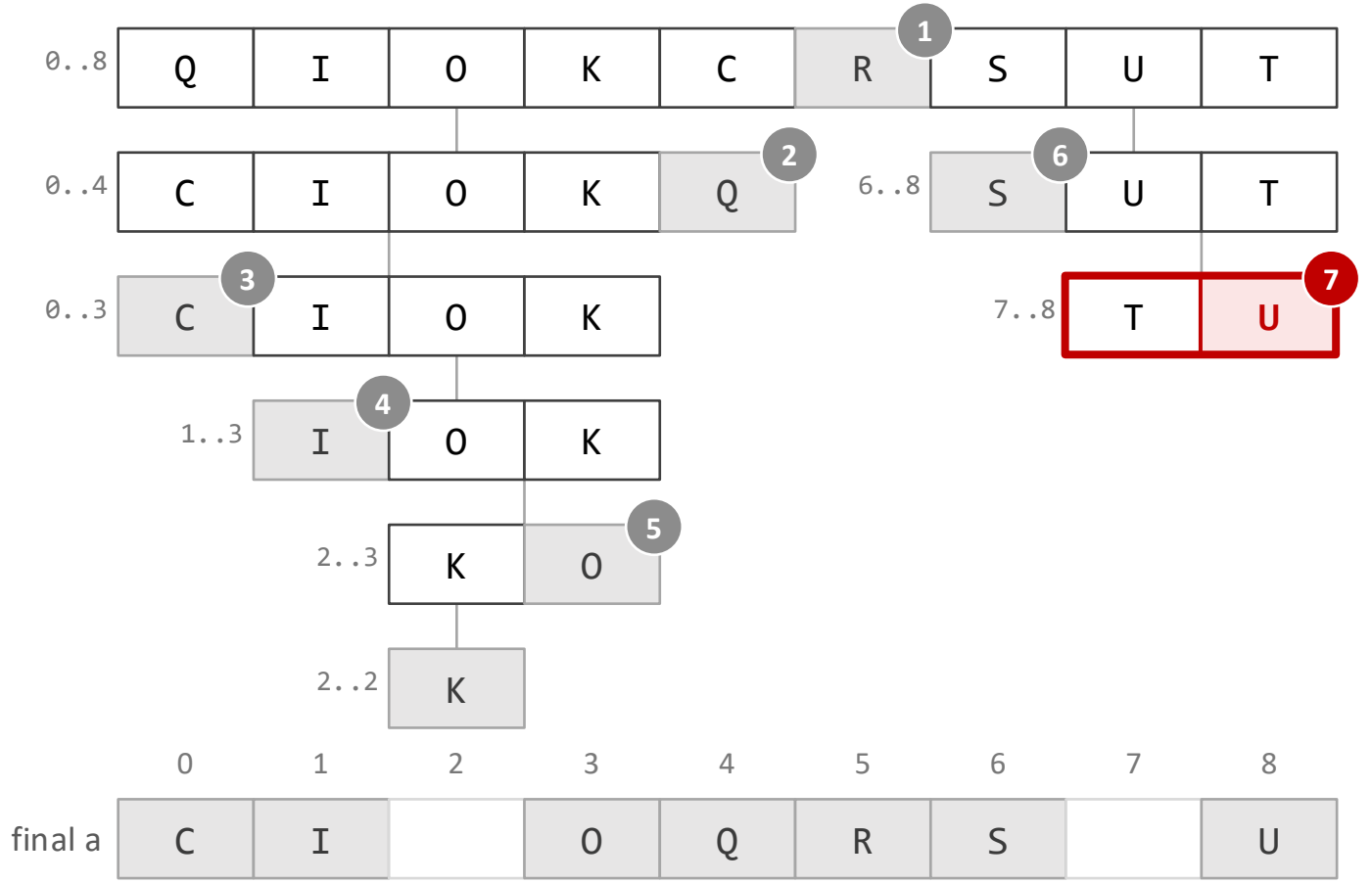
compares **30**

exchanges **8**

this partition: +4 compares, +1 exch.

Back up to the right piece from partition 1. **Partition 6:** `sort(a, 6, 8)`. **S** is the smallest; it stays put (swapped with itself, which still counts as an exchange).

Quicksort: the calls



cost of the whole sort so far

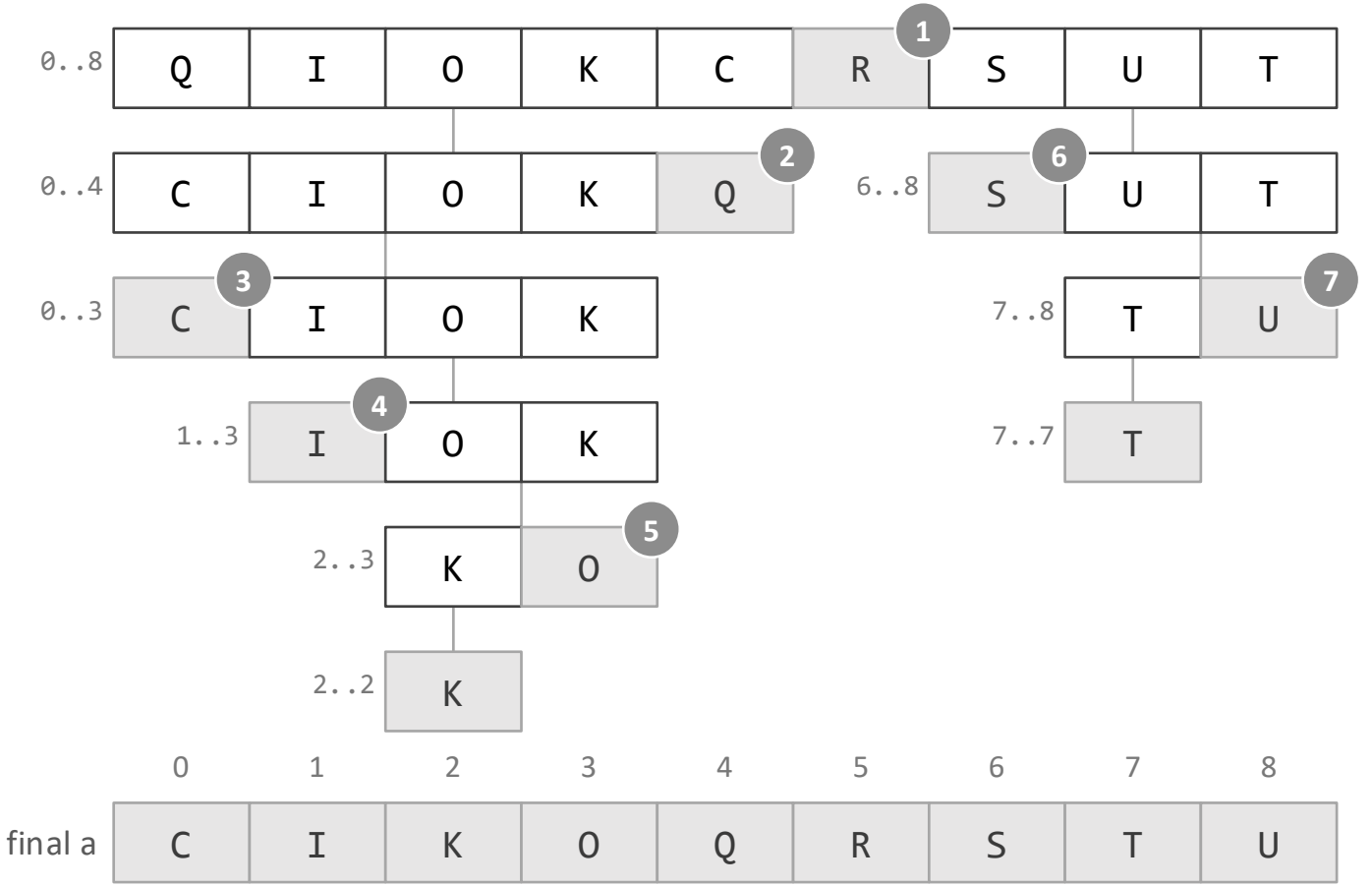
compares **32**

exchanges **9**

this partition: +2 compares, +1 exch.

Partition 7: `sort(a, 7, 8)`. U and T swap, leaving **T** as a piece of size 1.

Quicksort: the calls



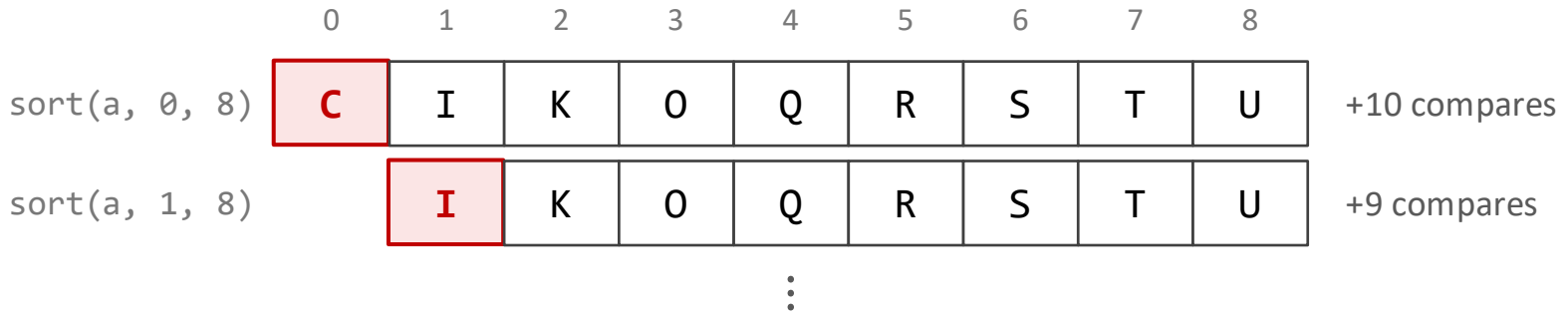
cost of the whole sort so far

compares	32
exchanges	9

7 partitions in all

Done: C I K O Q R S T U. 7 partitions, **32 compares, 9 exchanges**. Every item was placed as a pivot or left alone in a piece of size 1. No aux, no merging. (Empty calls like `sort(a, 0, -1)` aren't drawn; they return immediately.)

What if we skip the shuffle?

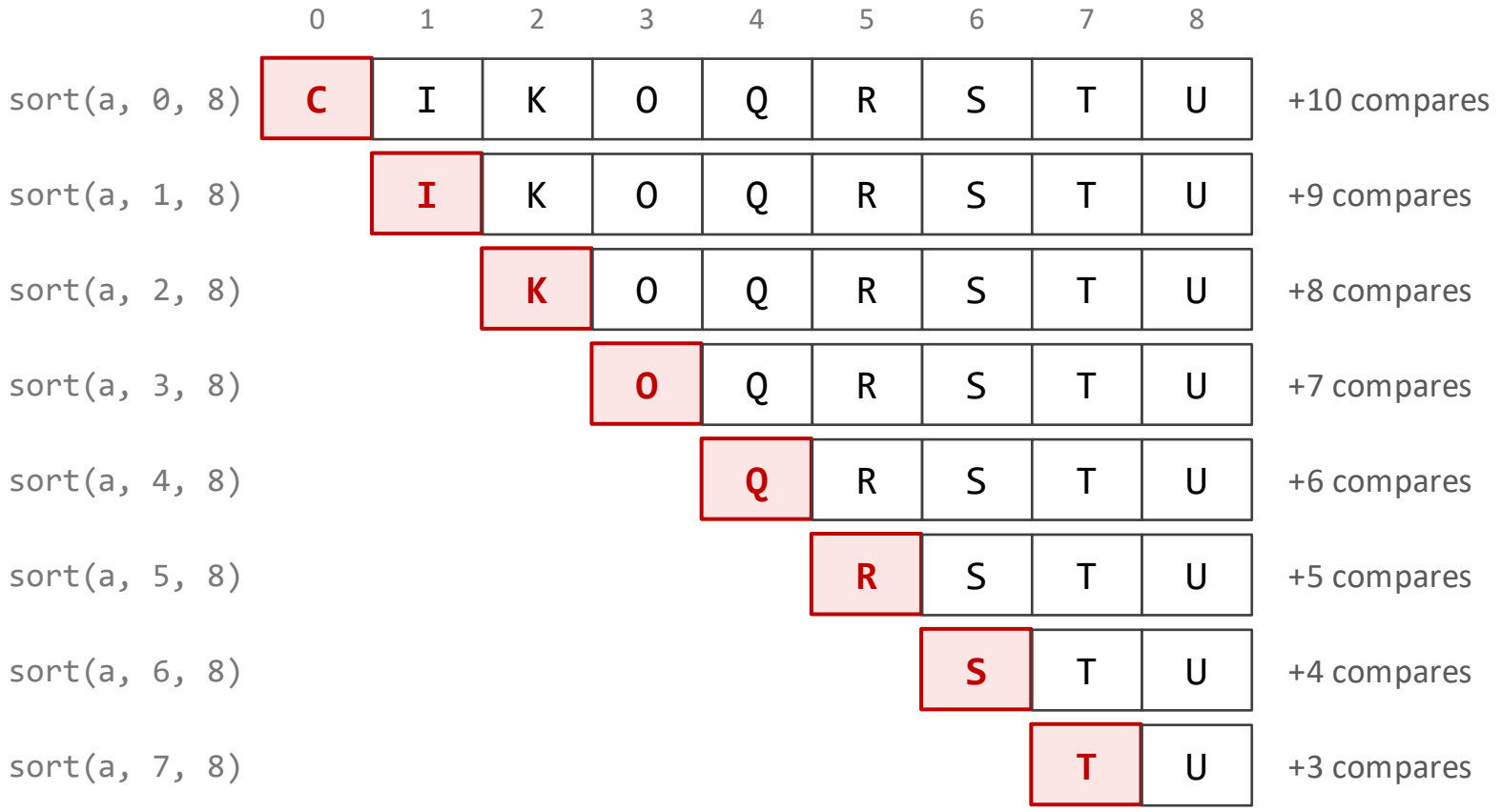


already-sorted input, no shuffle

compares	19
exchanges	2

Say the input is **already sorted**, and we skip the shuffle. The pivot **C** is the smallest item, so there's no left piece, and **everything else** goes right. Then **I** is the pivot, and the same thing happens.

What if we skip the shuffle?



already-sorted input, no shuffle

compares	52
exchanges	8

Every pivot is the smallest item in its piece, so each piece is just **one shorter** than the last. 8 partitions, **52 compares**, versus **32** for the shuffled version of the same letters.

How slow is that?

A piece of size n costs about n compares, and sizes go n, n − 1, n − 2, …, 2. So the total is about **$n^2/2$**: quadratic, like insertion sort.

n	**already sorted, no shuffle**	**shuffled first**	**no shuffle makes…**
9 (our letters)	52	32	1.6× as many
1,000	501,498	12,691	40× as many
10,000	50,014,998	174,864	286× as many
1,000,000	≈ 500,000,000,000 *	≈ 27,600,000 *	**≈ 18,000× as many**

compares, counted by running each sort (one shuffled run per row)

At a billion compares per second, a million already-sorted items takes about **8 minutes** instead of about **0.03 seconds**. Worse: the recursion goes **a million calls deep**, and Java runs out of stack space and crashes long before that.

* Estimated as $n^2/2$ and $2n \ln n$. At 10,000 items both formulas match the measured counts to within 6%.

Sorted input isn't rare

By accident

People sort data that's **already sorted** all the time: re-sorting a list after adding one item, or sorting data that came out of a database in order.

Reverse-sorted and **nearly sorted** input are just as bad for a leftmost pivot.

On purpose

If an attacker knows how you choose pivots, they can send input that hits the worst case **every time**, and tie up your server: a **denial-of-service** attack.

This isn't hypothetical. McIlroy (1999) showed how to build such inputs automatically against real library sorts.

So a fixed pivot rule (always the leftmost item, with no shuffle) is a bad bet for a library sort.

What the shuffle buys us

- After shuffling, the running time depends on **our random choices**, not on the order of the input. **No input is bad every time.**
- A user can't trigger the worst case by accident.
- An attacker can't force it either, **as long as they can't predict our random numbers**.
- The worst case is still **possible**: the shuffle could, by bad luck, line things up badly. For large n it's extraordinarily unlikely.

This is what **average case** means for quicksort: the average over the shuffles, for any input. Next time: what that average is, and just how unlikely the bad case is.

Is quicksort stable?

A sort is **stable** if equal items stay in the **same order they started in**. Here's our partition (no shuffle, so you can check it) sorting the roster by section:

1. Sorted by name

name	section
Adams	2
Baker	1
Chen	2
Diaz	1
Evans	2
Fox	1



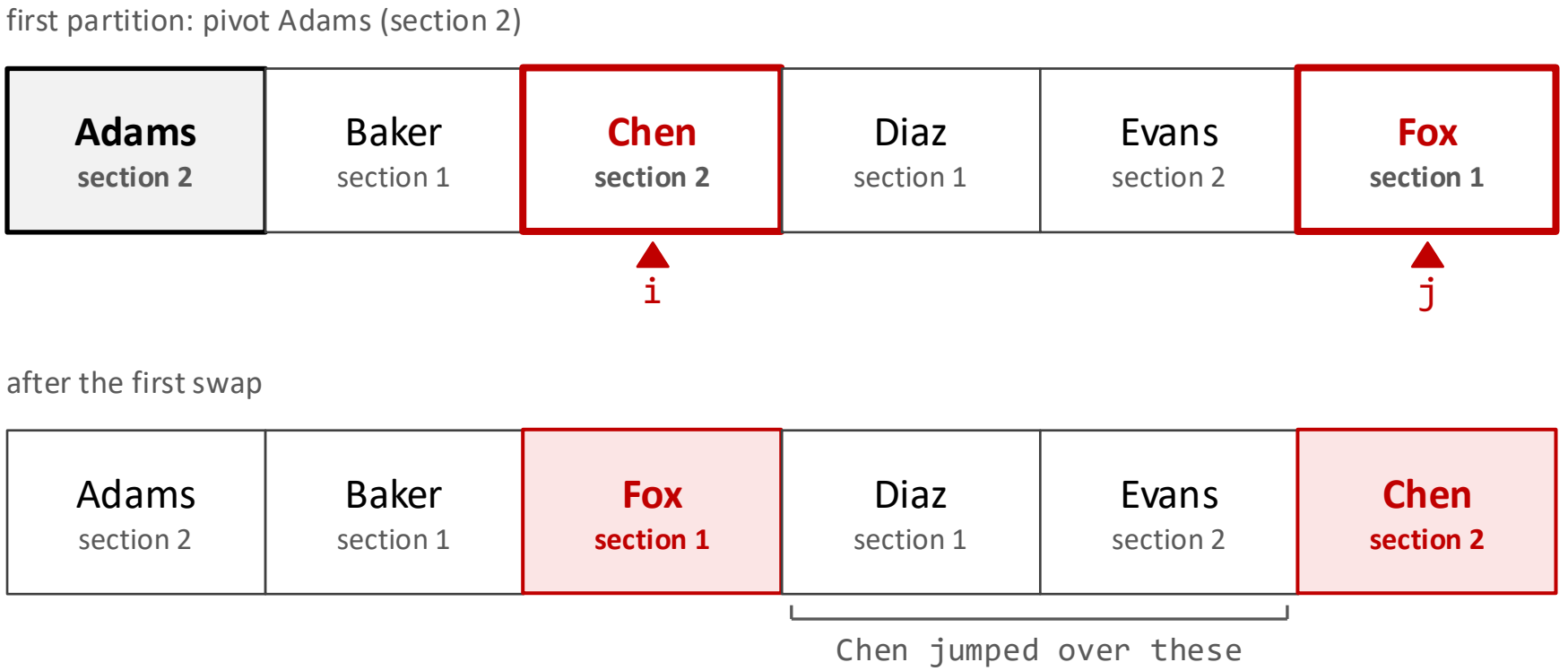
2. Then quicksort by section

name	section
Fox	1
Diaz	1
Baker	1
Evans	2
Adams	2
Chen	2

Sections are right, but names in red are **out of alphabetical order** within their section. Section 1 came out completely reversed.

Quicksort is not stable. With the shuffle it's worse: the scramble is different on every run.

Why: long-distance swaps



i stops at **Chen** and j stops at **Fox**, so they swap. Chen leaps past **Diaz and Evans**, and now comes after Evans, even though both are in section 2 and Chen came first. Nothing ever puts them back. **Mergesort never lets an item jump past an equal one:** on a tie, it takes from the left.

The trade-off

	insertion sort	**mergesort**	**quicksort**
compares, random input	~$n^2/4$	≤ n lg n	**~1.39 n lg n (average)**
worst case	~$n^2/2$	≤ n lg n	~$n^2/2$, but very unlikely after shuffling
extra memory	none	aux: n items	**none ***
stable?	yes	yes (ties go left)	**no**

* apart from the recursion itself, which (after shuffling) is usually only about lg n calls deep

On average quicksort makes **more** compares than mergesort, yet it's usually **faster**. **Next time:** why, and what happens when the data has lots of **duplicate keys**.